

WS73V100 常见问题

FAQ

文档版本 01

发布日期 2023-12-11

前言

概述

本文档主要介绍 WS73V100 解决方案中常见的问题处理和解决办法。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
WS73	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危

符号	说明
	害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
01	2023-12-11	第一次正式版本发布。

目 录

前言	i
1 SDK 开发环境和 SDK 使用类	1
1.1 SDK 开发环境问题	1
1.1.1 Python 版本不兼容冲突	1
1.1.2 依赖包问题	2
1.2 SDK 使用问题	3
1.2.1 Python 版本错误	3
1.2.2 SDK 编译出错未指定编译器	3
1.2.3 SDK 编译出错未指定编译方式 (GCC or CLANG)	4
2 系统异常类	6
2.1 Host 侧和 Device 侧通信异常	6
2.1.1 SDIO 设备通信异常	6
2.1.2 USB 设备通信异常	7
2.1.3 FRW 消息传输异常	8
3 软件常见问题及定位方法	9
3.1 驱动加载失败	9
3.1.1 提示: insmod: short read	9
3.1.2 提示: Exec format error	9
3.1.3 提示: Firmware 下载失败	10
3.1.4 提示: Unknown symbol cfg80211_xxx	11
3.1.5 提示: Unknown symbol hci_free_dev	12
3.2 wpa_supplicant 运行失败	13
3.3 DebugKits 使用失败	13

1 SDK 开发环境和 SDK 使用类

本章节主要介绍 SDK 开发环境搭建以及使用时出现的常见问题处理和解决办法。

1.1 SDK 开发环境问题

1.2 SDK 使用问题

1.1 SDK 开发环境问题

- 请参考《WS73V100 Linux 平台驱动移植 用户指南》中“Python 环境安装”章节，安装 SDK 依赖 Python 及 Python 库。
- 本章节中图例为说明问题使用的测试代码，与实际使用版本有差异，以构建脚本要求版本为准。

1.1.1 Python 版本不兼容冲突

问题描述

图1-1 在 SDK 下执行 make 失败

```
$ make BLE_ANDROID=y
File "/home/xxx/xxx/xxx/build/scripts/mconfig.py", line 18
    self._old_dir: str = os.getcwd()
    ^
SyntaxError: invalid syntax
make: *** [prepare] Error 1

#### make failed to build some targets ####
```

在 SDK 中执行“make xxx”命令，报错：SyntaxError: invalid syntax。

问题分析

Python 3.8 引入了海象运算符 (Walrus Operator)，它使用符号 “:=” 表示，它可以将表达式的值赋给变量，并且在同一表达式中使用该变量。

若当前使用的 Python 版本低于 3.8，会导致无法识别符号 “:=” 而报错。

解决方法

安装 Python 3.8+版本。

- 以命令行安装：

```
sudo apt-get install python3.8
```

- 源码包编译安装：

```
tar -xf Python-3.8.5.tgz
cd Python-3.8.5
./configure
sudo make
sudo make install
```

1.1.2 依赖包问题

问题描述

依赖包版本与实际使用版本不一致，如图 1-2 所示。

图1-2 Python 依赖包不匹配

```
scons: Reading SConscript files ...
3.7.3 (default, Dec 27 2019, 03:17:52)
[GCC 5.4.0 20160609]
ModuleNotFoundError: No module named 'Crypto':
  File "/home/xubinhua/Hi3861V100R001C01SPC020B030/SConstruct", line 7:
    from scripts import common_env, scons_utils, scons_app, scons_env_cfg, pkt_builder
  File "/home/xubinhua/Hi3861V100R001C01SPC020B030/build/scripts/pkt_builder.py", line 12:
    import make_upg_file as MAKE_IMAGE
  File "/home/xubinhua/Hi3861V100R001C01SPC020B030/build/scripts/make_upg_file.py", line 13:
    from Crypto.Hash import SHA
```

问题分析

Pycryptodome 未安装，或者安装的版本与 Python 版本不匹配而引发的问题时常发生，因为 Linux 系统允许安装多个 Python 版本，依赖包根据不同 Python 版本安装后，不会共享依赖包。用户在安装依赖包时，容易使用错误的 Python 版本安装。

解决方法

SDK 中依赖包需要支持 python3.8 版本，确认已安装依赖包与编译要求的 Python 版本相匹配。如果没有安装，请使用 python3.8 安装。

1.2 SDK 使用问题

1.2.1 Python 版本错误

问题描述

引发报出 Python 版本错误的因素有多种，例如：“1.1 SDK 开发环境问题”中介绍的安装库版本不匹配问题。以下提供一些常用的排查思路和解决方法。

解决方法

通过“python --version”查看默认 Python 版本（要求为 python3.8+）。

- 如果没有安装 python3.8，需先安装。
- 如果无法修改默认 Python 默认使用版本，需在个人目录下，建立 python3.8 的软链接（例如：~/bin），将此目录添入“PATH”系统参数中，并且此目录路径排在 PATH 最前，重新登陆终端，再次查看 Python 版本。（如把当前目录添加到环境变量，并且优先使用，export PATH=\$(pwd):\$PATH）

1.2.2 SDK 编译出错未指定编译器

问题描述

编译 SDK 版本时，上报类似编译链、内核路径不存在等问题，如图 1-3 所示。

图1-3 编译器配置异常示例

```
@yanta:~/sdk_ws73/tmp$ make all
kconfig header saved to '/root/.sdk_ws73/output/bin/autoconfig.h'
INI file generate success in /root/.sdk_ws73/output/bin/ws73_cfg.ini!
cd /root/.sdk_ws73/output/bin && \
BUILD_DEVICE_WITH_ROM_REPO=yes\
make -j48 && cp -f plat_soc.ko /root/.sdk_ws73/output/bin/
make[1]: Entering directory '/root/.sdk_ws73/output/bin/driver/platform'
make -C "" ARCH="arm" CROSS_COMPILE="" M=/root/.sdk_ws73/output/bin/driver/platform modules -j KBUILD_MODPOST_WARN=1
make: Entering an unknown directory
```

SDK 编译前会检查编译器是否已经指定正确。出现错误的情况有：

- 未将编译器路径加到\$PATH 中，或者指定不正确。
- \$PATH 指定了多款编译器路径，导致检查出错。
- 编译器未完整安装。编译器 bin 目录中不包含 “arm-unknown-linux-gcc”。

解决方法

检查环境变量\$PATH，将编译器路径添加到服务器环境变量。

编译器检查有以下条件：

- 路径以 “bin” 或 “bin/” 结尾。
- bin 目录中需包含 “arm-unknown-linux-gcc”。
- 尽量勿将多个版本的编译器放在同一路径下，因为系统默认使用第一个被找到的编译器，可能与所需要版本不匹配。

图1-4 编译器配置正常示例

```
#
# build
#
# Configure the compilation environment
#
WSCFG_CROSS_COMPILE="arm-himix100-linux-"
WSCFG_KERNEL_DIR="/root/.sdk_ws73/output/bin/kernel/linux-4.9.y"
WSCFG_ARCH_ARM=y
# WSCFG_ARCH_CUSTOM is not set
WSCFG_ARCH_NAME="arm"
BOARD_ASIC=y
BOARD_ASIC_WIFI=y
WSCFG_BUS_SDIO=y
```

1.2.3 SDK 编译出错未指定编译方式（GCC or CLANG）

问题描述

编译 SDK 时会检查是否指定了编译方式，如果未添加 WSCFG_USING_GCC 及 WSCFG_USING_LLVM_CLANG 选项，会出现编译问题，如图 1-5 所示。

图1-5 未配置编译方式示例

```
hi3873-master$ make
warning: CONFIG_PLAT_DFR_OUTPUT_PATH (defined at driver/platform/kconfig:54) has leading or trailing whitespace in its prompt
No change to Kconfig header in /hi3873-master/output/bin/autoconfig.h'
warning: CONFIG_PLAT_DFR_OUTPUT_PATH (defined at driver/platform/kconfig:54) has leading or trailing whitespace in its prompt
INI file generate success in /home/x00383584/code/TR5/hi3873-master/output/bin/ws73_cfg.ini!
cd /hi3873-master/driver/platform && \
  BUILD_DEVICE_WITH_ROM_REPO=yes \
  make -j48 && cp -f plat_soc.ko /hi3873-master/output/bin/
make[1]: Entering directory '/hi3873-master/driver/platform'
Makefile:382: *** "error please specify an available compiler!". Stop.
make[1]: Leaving directory '/hi3873-master/driver/platform'
make: *** [Makefile:87: platform] Error 2
x00383584@yanfa:~/code/TR5/hi3873-master$
```

SDK 编译前会检查编译方式是否已经指定正确。出现错误的情况有：

- 既未指定为 GCC 编译模式又未指定未 CLANG 编译模式。
- 指定了 GCC 模式，但是未配置工具链。
- 指定了 CLANG 模式，但是未配置 CLANG PATH。

解决方法

正确指定编译方式，并且指定配套的编译工具路径，如下示例图 1-6 及：

图1-6 GCC 配置正确示例

```
# Generate by menuconfig
#
# build
#
#
# Configure the compilation environment
#
WSCFG_USING_GCC=y
# WSCFG_USING_LLVM_CLANG is not set
WSCFG_CROSS_COMPILE="arm-linux-gnueabihf" toolchains/gcc-linaro-6.3.1/bin/arm-linux-gnu-
WSCFG_KERNEL_DIR="/hi3873-master/output/bin/kernel/KERNEL_OBJ"
```

图1-7 CLANG 配置正确示例

```
# Generate by menuconfig
#
# build
#
#
# Configure the compilation environment
#
# WSCFG_USING_GCC is not set
WSCFG_USING_LLVM_CLANG=y
WSCFG_CROSS_COMPILE="arm-linux-gnueabihf" gcc-arm-linux-gnu-5.5.0-ubuntu/x86_64/bin/arm-linux-gnu-
WSCFG_CLANG_PATH="/hi3873-master/output/bin/clang-r450784e/bin"
WSCFG_KERNEL_DIR="/hi3873-master/output/bin/kernel/KERNEL_OBJ/5.15_13"
WSCFG_EXTRA_FLAGS=""
WSCFG_EXTRA_PARAMS="LLVM=1 LLVM_IAS=1"
# WSCFG_ARCH_ARM is not set
```

2 系统异常类

本章节主要介绍异常触发时看门狗相关问题和异常导致死机信息打印的相关问题。

2.1 Host 侧和 Device 侧通信异常

2.1 Host 侧和 Device 侧通信异常

2.1.1 SDIO 设备通信异常

问题描述

加载 plat_soc.ko 时，显示 “sdio load fail”，如图 2-1 所示。

图2-1 SDIO 加载异常

```
/komod # insmod plat_soc.ko
plat_soc: loading out-of-tree module taints kernel.
plat_soc:EMERG]0:emerg log.
plat_soc:ALERT]1:alert log.
plat_soc:CRIT]2:crit log.
plat_soc:E]3:err log.
plat_soc:W]4:warning log.
plat_soc:E]custom ini[plat kern log level]: 7.
plat_soc:E]board_power_gpio_init::succ, power_gpio_idx=[40]!
plat_soc:E]board_wakeup_gpio_init::succ, wkup_gpio_idx=[70]!
plat_soc:E]tty unlocked_ioctl() ret: 0
plat_soc:E]zdiag uart rx task enter
plat_soc:E]tcp rx task

[SDIO][E]sdio load fail.
[sdio_trigger_probe:2215]
[SDIO][E]sdio first probe failed![hcc_adapt_sdio_load:2547]
plat_soc:E]hcc init: fail
plat_soc:E][SH_PROC] task exit.
plat_soc:E]zdiag_uart: channel exit enter.
```

解决方法

1. 检测硬件焊接，SDIO 设备与主控设备焊接是否正常。
2. 执行“cat /proc/mci/mci_info”命令，查看是否可以识别到 SDIO 设备。
3. 检查“Host work clock”配置频率，不能超过 50MHz。

图2-2 SDIO 设备示例

```
~ # cat /proc/mci/mci_info
MCI0: plugged_disconnected
MCI1: plugged_connected
      Type: SDIO card Mode: HS
      Speed Class: Class 0
      Uhs Speed Grade: Less than 10MB/sec(0h)
      Host work clock: 50MHz
      Card support clock: 50MHz
      Card work clock: 50MHz
      Card error count: 0
MCI2: invalid
```

2.1.2 USB 设备通信异常

问题描述

加载 plat_soc.ko 驱动时，加载失败。。

解决方法

WS73 模组默认 ID 号为 ffff:3733，执行 “lsusb”，若主控未识别到相关设备，检测设备连接是否异常。

图2-3 WS73 USB 模组示例

```
/komod # lsusb
Bus 001 Device 001: ID 1d6b:0002
Bus 001 Device 002: ID ffff:3733
Bus 002 Device 001: ID 1d6b:0003
```

2.1.3 FRW 消息传输异常

问题描述

成功加载驱动后，显示消息传输超时，如图 2-4 所示。

图2-4 frw 消息超时

```
44.555781] [BT_DEV_INFO] bt_misc_dev_open:111
45.397918] frw_host_post_sync: thread id 0 TIMEOUT! msg_id 85 seq 104
45.404708] {frw_thread_dump:: thread_name: wifi_frwrmsg, is_running: 1}
45.411697] {frw_thread_dump:: queue_id:0, msg_cnt:5}
45.416881] {frw_thread_dump:: queue_id:2, msg_cnt:12}
45.422234] {frw_thread_dump:: thread_name: wifi_frwr_txdata, is_running: 0}
45.429455] {hmac_vap_sync::vap[2] sync msg failed[203].}
45.435536] Oh no,LinkLoss!! BSSID[A8:5A:E0:E8:XX:XX]
45.442835] hmac_single_hal_device_scan_complete:vap[2] time[10005] chan_cnt[13] chan_0[1] back[1] event[7] mode[11]
46.202284] input:  as /devices/virtual/input/input11
47.445905] frw_host_post_sync: thread id 0 TIMEOUT! msg_id 357 seq 41
47.455142] {frw_thread_dump:: thread_name: wifi_frwrmsg, is_running: 1}
47.462535] {frw_thread_dump:: queue_id:0, msg_cnt:5}
47.467912] {frw_thread_dump:: queue_id:2, msg_cnt:13}
47.473980] {frw_thread_dump:: thread_name: wifi_frwr_txdata, is_running: 0}
47.509917] frw_host_post_sync: thread id 0 TIMEOUT! msg_id 85 seq 105
47.516708] {frw_thread_dump:: thread_name: wifi_frwrmsg, is_running: 1}
47.523883] {frw_thread_dump:: queue_id:0, msg_cnt:5}
47.529113] {frw_thread_dump:: queue_id:2, msg_cnt:13}
47.534416] {frw_thread_dump:: thread_name: wifi_frwr_txdata, is_running: 0}
47.541811] {hmac_vap_sync::vap[2] sync msg failed[203].}
```

问题分析

驱动 Host 侧和 Device 侧传输的 MSG 消息 ID 不匹配，导致通信异常。

解决方法

检测驱动 ko 和 firmware 文件是否为同一 SDK 版本。

3 软件常见问题及定位方法

3.1 驱动加载失败

3.2 wpa_supplicant 运行失败

3.3 DebugKits 使用失败

3.1 驱动加载失败

3.1.1 提示: insmod: short read

问题描述

insmod 驱动加载失败, 提示 "insmod: short read"。

问题分析

加载驱动时, 提示 "insmod: short read", 通常是 ko 文件下载不完整导致。

解决方法

检查下载 ko 到板端的工具是否正常运行、链路是否联通。确定链路联通状态下, 重新下载驱动到板端。

3.1.2 提示: Exec format error

问题描述

insmod 驱动加载失败, 提示 "Exec format error"。

问题分析

Exec format error 通常是由于尝试运行不兼容的二进制文件或脚本文件而引起的。此外，操作系统架构和二进制文件架构不匹配也可能导致此错误。例如将 32 位交叉编译器生成的二进制文件放到 64 位系统上运行会导致 Exec format error。

解决方法

在目标开发板上输入 “uname -a” 查看板端系统信息；在 SDK 编译服务器上输入 “file xxx.ko” 查看驱动文件信息。检测两者 OS 版本、CPU 架构是否一致。

3.1.3 提示：Firmware 下载失败

问题描述

加载 wifi_soc.ko 时，下载固件时，无法找到对应 bin 文件，导致驱动加载失败，如图 3-1 所示。

图3-1 wifi 驱动加载失败

```
/komod # insmod wifi_soc.ko
[FRW] frw_timer_init enter
frw_start_hcc_service type:6, ret=0
frw_start_hcc_service type:4, ret=0
[FRW] frw_main_init etc end
plat_soc:E]check if var and value is NULL or blankplat_soc:E]check if var and value is NULL or blank
hwifi_custom_host_read_cfg_init finish!
hwifi_cfg_host_global_init_param: rx stbc[1] ret[0]
hwifi_cfg_tcp_ack_param_init: tcp_ack_filter_enable[1] ret[0]
hwifi_cfg_tcp_ack_param_init: tcp_ack_max_num_start_process[2] ret[0]
hwifi_cfg_host_global_init_param: user num[8] ret[0]
hwifi_cfg_scan_probe_ie_cfg_init ret 0 priv_value 1
hwifi_cfg_scan_probe_del_wps_ie_cfg_init ret 0 priv_value 1
g_scan_probe_req_del_wps_ie = 1
hwifi_cfg_random_mac_addr_scan_init ret 0 priv_value 1
hwifi_cfg_random_mac_addr_scan_oui_init ret 65535 priv_value 0
hwifi_cfg_pcap_file_len_max_cfg_init ret 0 priv_value 12
hwifi_cfg_data_sample_param_init: data_sample[0] ret[0]
hwifi_cfg_host_global_init_param: apf enable[1] ret[0]
hwifi_cfg_host_global_init_param: llax protocol enable[0] ret[0]
hwifi_cfg_host_global_init_param: wow_event[15] ret[0]
hwifi_cfg_host_global_init_param: wow_enable[1] ret[0]
hwifi_cfg_host_global_init_param: smooth_phase_en[1] ret[0]
hwifi_cfg_host_global_init_param: over_ds_en[1] ret[0]
dfx_user_rd_init success!
hmac_register_pm_callback_etc:wlan_pm_get_wifi_srv_handler_etc is null
plat_soc:E]pm_svc_open::first svc open, pm_svc_power_on!
[SDIO][I]sdio state changed, tx[off=>on ],rx[off=>on ][sdio_print_state:284]
plat_soc:E]firmware download etc begin
plat_soc:E]filp open [/etc/ws73/ws73.bin] fail!!, fp= (null)
plat_soc:E][[/etc/ws73/ws73.bin] sha256sum check fail!!
plat_soc:E]parse file fail!
plat_soc:E]download firmware fail
plat_soc:W]download firmware, count [1], current time [19511]us, max time [19511]us
plat_soc:E]wait bsp ready timeout 1000 ms
```

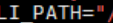
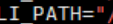
问题分析

wifi 驱动加载时，无法找到对应的 bin 文件，导致通信设备。

解决方法

在 SDK 根目录下，修改.config 文件，编辑 firmware 文件路径。

图3-2 修改.config 文件文件路径

```
#  
# Configure the loading path of files such as firmware  
#  
CONFIG_FIRMWARE_BIN_PATH="/etc/firmware/ws73.bin"  
CONFIG_FIRMWARE_WIFICALI_PATH="/etc/firmware/wifi_cali.bin" 固件存放位置  
CONFIG_FIRMWARE_BSLECALI_PATH="/etc/firmware/btc_cali.bin"  
CONFIG_FIRMWARE_WOW_PATH="/etc/firmware/wow.bin"  
CONFIG_INI_FILE_PATH="/etc/firmware/ws73_cfg.ini" ini文件位置  
# end of Configure the loading path of files such as firmware
```

3.1.4 提示：Unknown symbol cfg80211_xxx

问题描述

insmod wifi_soc.ko 失败，提示 “Unknown symbol cfg80211_xxx”，如图 3-3 所示。

图3-3 未识别 cfg80211 相关符号

```
/komod # insmod wifi_soc.ko  
wifi_soc: Unknown symbol __cfg80211_alloc_event_skb (err 0)  
wifi_soc: Unknown symbol wiphy_register (err 0)  
wifi_soc: Unknown symbol cfg80211_remain_on_channel_expired (err 0)  
wifi_soc: Unknown symbol cfg80211_vendor_cmd_reply (err 0)  
wifi_soc: Unknown symbol cfg80211_del_sta_sinfo (err 0)  
wifi_soc: Unknown symbol wiphy_unregister (err 0)  
wifi_soc: Unknown symbol cfg80211_connect_bss (err 0)  
wifi_soc: Unknown symbol cfg80211_ch_switch_notify (err 0)  
wifi_soc: Unknown symbol cfg80211_ready_on_channel (err 0)  
wifi_soc: Unknown symbol __ieee80211_get_channel (err 0)  
wifi_soc: Unknown symbol wiphy_free (err 0)  
wifi_soc: Unknown symbol wiphy_new_nm (err 0)  
wifi_soc: Unknown symbol cfg80211_get_bss (err 0)  
wifi_soc: Unknown symbol __cfg80211_send_event_skb (err 0)  
wifi_soc: Unknown symbol cfg80211_disconnected (err 0)  
wifi_soc: Unknown symbol cfg80211_michael_failure (err 0)  
wifi_soc: Unknown symbol cfg80211_scan_done (err 0)
```

问题分析

在 insmod wifi_soc.ko 前，cfg80211 模块相关符号未注册给 Linux 内核。

解决方法

请参见《WS73V100 Linux 平台驱动移植 用户指南》中的“驱动加载出错”章节。

3.1.5 提示：Unknown symbol hci_free_dev

问题描述

insmod ble_soc.ko 失败，提示 “Unknown symbol hci_free_dev”，如图 3-4 所示。

图3-4 内核缺少蓝牙模块

```
[15403.297093] ble_soc: Unknown symbol hci_free_dev (err -2)
[15403.297473] ble_soc: Unknown symbol hci_alloc_dev (err -2)
[15403.297750] ble_soc: Unknown symbol hci_unregister_dev (err -2)
[15403.298017] ble_soc: Unknown symbol hci_rcv_frame (err -2)
[15403.298240] ble_soc: Unknown symbol hci_reset_dev (err -2)
[15403.298440] ble_soc: Unknown symbol hci_register_dev (err -2)
[15403.337373] ble_soc: Unknown symbol hci_free_dev (err -2)
[15403.337746] ble_soc: Unknown symbol hci_alloc_dev (err -2)
[15403.338077] ble_soc: Unknown symbol hci_unregister_dev (err -2)
[15403.338327] ble_soc: Unknown symbol hci_rcv_frame (err -2)
[15403.338558] ble_soc: Unknown symbol hci_reset_dev (err -2)
[15403.338726] ble_soc: Unknown symbol hci_register_dev (err -2)
```

问题分析

内核编译未开启蓝牙相关选项。

解决方法

重新编译 linux 内核，开启蓝牙相关选项。

- 1、进入内核编译目录，执行 make ARCH=arm CROSS_COMPILE=arm-himix100-linux- menuconfig 进入配置页。
- 2、进入 Networking support -> Bluetooth subsystem support -> Bluetooth Low Energy (LE) features 选项，输入 y 勾选。
- 3、重新编译内核镜像后烧录。

说明

ARCH 为相应的处理器架构

CROSS_COMPILE 为相应的编译链信息

3.2 wpa_supplicant 运行失败

问题描述

执行 “wpa_supplicant -iwlan0 -Dnl80211 -c/etc/wpa_supplicant.conf &” 命令后，无法创建 socket 程序和后台 wpa_supplicant 通信。

解决方法

图3-5 wpa_supplicant.conf 配置文件示例

```
ctrl_interface=/var/run/wpa_supplicant  #必须配置
update_config=1                          #强制更新覆盖配置
```

建立 “wpa_supplicant” 通信，可依次排查下列场景：

1. 为 wpa_supplicant 添加可执行权限。
2. 执行 “wpa_supplicant” 命令时，确定指定路径下存在 “wpa_supplicant.conf” 配置文件，如上述命令中需保证 “/etc” 目录下，存在 “wpa_supplicant.conf” 文件。
3. “wpa_supplicant.conf” 文件 “ctrl_interface” 会记录对应 socket 端口存放的位置，确保该目录下具有读写权限。

3.3 DebugKits 使用失败

问题描述

DebugKits 工具连接失败。

解决方法

请参见《WS73V100 DebugKits 工具 使用指南》中 “连接设置” 章节。